CSE_3523

Segment-5

Microprocessor, microcontroller and embedded system

Course code: CSE-3523

Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer

Dept. of CSE, IIUC

Mujiburmaruf.cuet17@gmail.com

International Islamic University Chittagong(IIUC)

Fundamentals of Computing

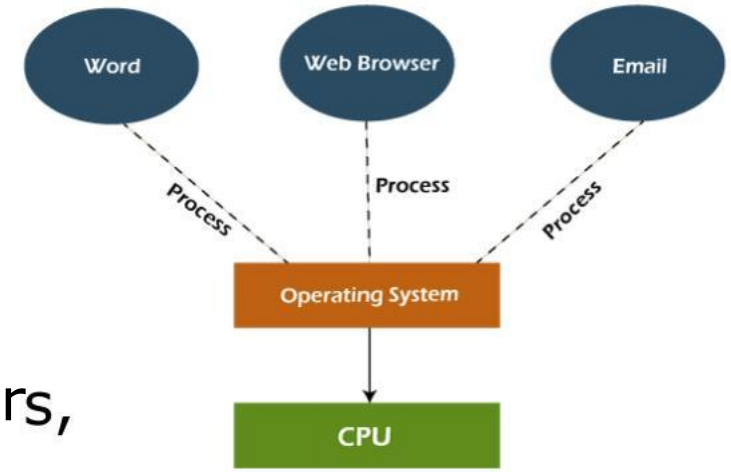
Multitasking

Process and thread

Multi-user System

Multitasking:

- **Multitasking** is the ability of an operating system to execute multiple tasks (also called *processes* or *programs*) seemingly at the same time.
- This doesn't mean all tasks run *exactly* at the same time, but the CPU switches between them so quickly that it appears simultaneous to the user.



Key Concepts:

- **Context Switching:**
  - The CPU saves the state of the current task (registers, memory pointers) and loads the state of the next task.
  - Enables efficient use of CPU time.

Multitasking:

- **Automatic Task Handling:**
  - The system interrupts the current program automatically, saves its state, and resumes another task.
  - This switching is handled by the OS without user interference.

multitasking systems, the CPU gives each process a small time slice. It switches tasks so efficiently that users don't notice interruptions. It's like juggling — only one ball is in hand at a time, but all seem active.

Real-World Examples:

Downloading a file while watching a video

Updating software while browsing the internet

Running antivirus scans while working on spreadsheets

Multi user System

A **multi-user system** is a computer system that allows **multiple users** to access and **use computing resources simultaneously or in turns**. These systems manage and isolate user activities, ensuring that one user's actions do not interfere with others.

Types of Multi-User Systems:

Time-Sharing Systems:CPU time is divided into small time slices and distributed among users.

Networked Systems: Multiple users connect to a central server over a network

Examples:

UNIX/Linux servers used in universities or companies

Mainframe computers in banking or government sectors

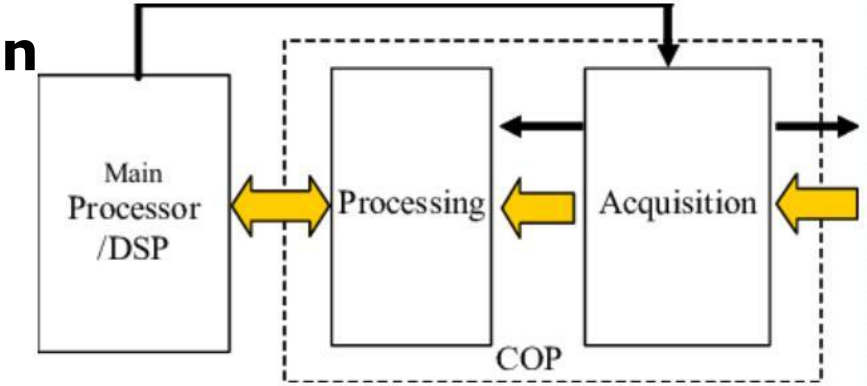
Remote desktop systems or virtual machines (VMs)

Performance Enhancers in Modern CPUs

- **Coprocessor**
- **Dual core Processor**
- **Cache memory**

Coprocessor:

A **coprocessor** is a *helper processor* that works with the **main CPU** to handle specific jobs more efficiently.



Why It's Used:

Think of the CPU as a *team leader* — it manages many tasks. But when tasks become *too complex or repetitive*, the CPU hands them off to a **coprocessor** — a *specialist* that does the job faster and better.

Main Purpose:

To **speed up performance** by handling:

1.Heavy calculations 2.Graphics rendering 3.Signal processing

Real Benefit:

Makes computer **run faster and smoother** by letting the CPU focus on general work, while the coprocessor handles the heavy lifting.

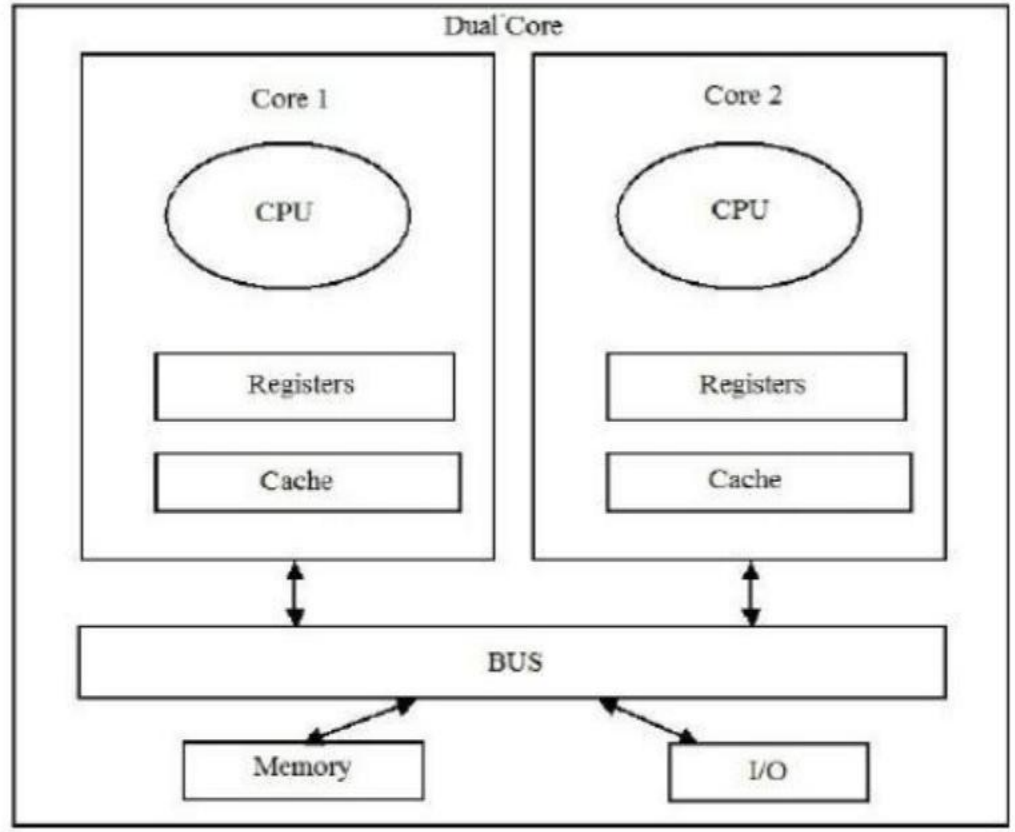
dual core Processor

Dual Core Processor

A **dual-core processor** is a single chip that contains **two independent CPU cores**. Each core can **execute tasks separately**, enabling **parallel processing** and improved **multitasking performance**.

Example:

Intel Core i3-2100: Dual-core processor from Intel’s 2nd generation lineup



Cache memory

Cache Memory is a **small, high-speed memory** located close to the CPU that stores **frequently accessed data and instructions**.

Why Is Cache Memory Important?

Modern processors operate **much faster than main memory (RAM)**. If the CPU had to wait for data to be fetched from RAM every time, it would **slow down system performance significantly**. Cache memory reduces this delay by storing copies of frequently used data, allowing the CPU to access it **almost instantly**.

Characteristics:

Much faster than RAM

Smaller in size (usually measured in KB or MB)

Volatile memory (data is lost when power is off)

Types of Cache Memory

L1 Cache (Level 1):

Fastest and smallest

Integrated directly into the CPU chip

Stores very frequently accessed data

Size: Typically 16 KB to 128 KB

L2 Cache (Level 2):

Larger but slightly slower than L1

May be on the CPU chip or a separate chip

Stores recently used data

Size: 256 KB to 1 MB+

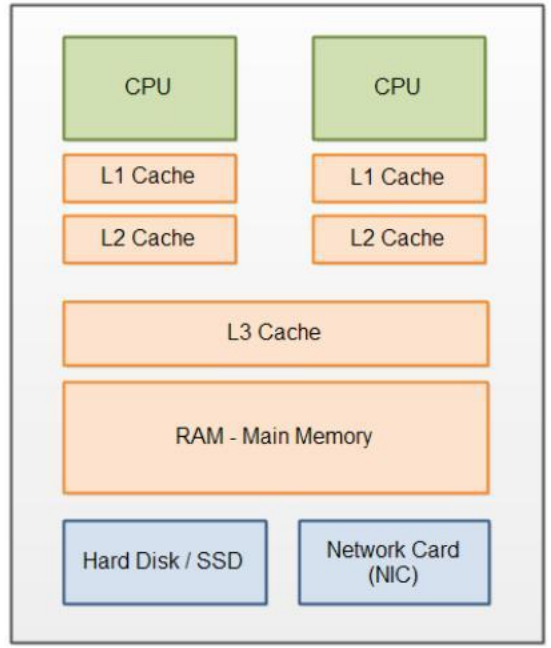
L3 Cache (Level 3):

Shared among cores in multi-core processors

Slower than L1 and L2 but faster than RAM

Helps improve overall performance

Size: 2 MB to 32 MB+

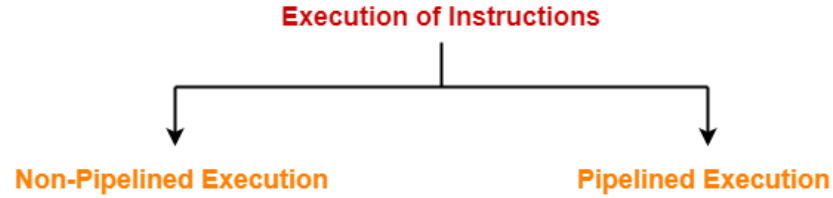


Instruction Execution Types

Pipelining

Instruction Execution Types

In computer architecture, instruction execution is classified into **two types** based on how the CPU processes instructions: **Non-Pipelining** and **Pipelining**. This depends on whether instructions are executed **sequentially** or **overlapped**, directly affecting CPU performance and efficiency.



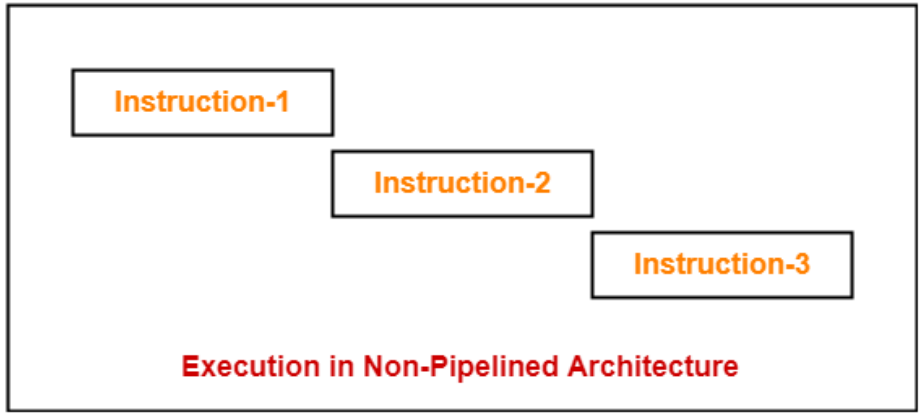
1. Non-Pipelining Architecture

Executes **one instruction at a time**

All stages (fetch → decode → execute → memory → write-back) must **finish before the next instruction starts**

Example:

If one instruction takes **5 clock cycles**, the next instruction starts **only after** those 5 cycles finish (non-pipelined).



What is Pipelining?

- Pipelining refers to dividing the execution of an instruction into multiple stages and executing these stages in parallel for different instructions. This approach reduces idle time of processor components and increases instruction throughput.

Key points:

- Instructions are divided into multiple stages
- Next instruction starts before previous finishes
- Improves hardware utilization
- **Pipeline Structure**
- Each stage has **input register + logic**
- Registers store **intermediate results**
- Output of one stage → **input of next**
- Ensures **synchronized data flow**

- **Example:**
- Factory: one car is painted, another assembled, and another inspected simultaneously.

Instruction Pipeline Stages

A typical instruction pipeline consists of five stages. Each stage performs a specific function and allows different instructions to be processed concurrently.

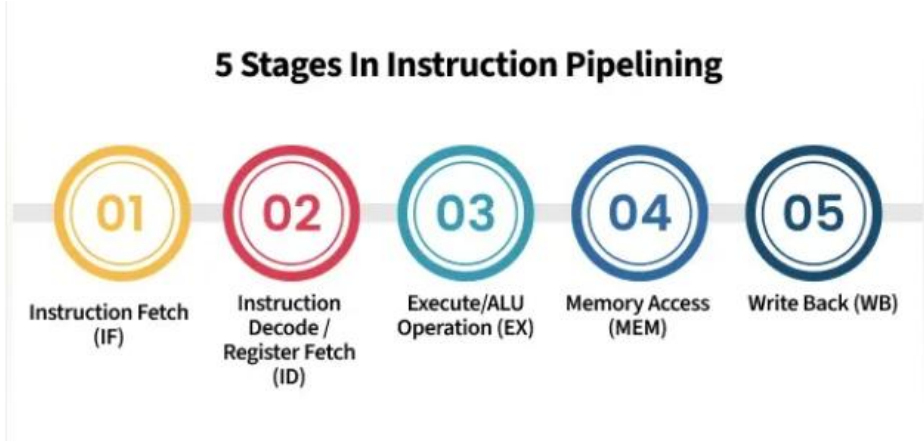
IF (Instruction Fetch): Fetch instruction from memory

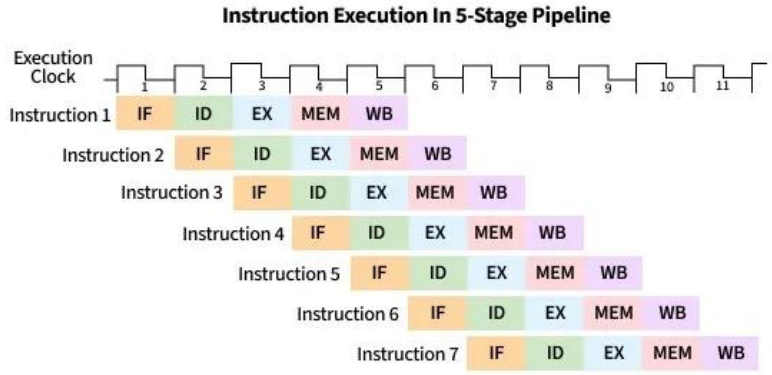
ID (Instruction Decode): Decode instruction, read registers

EX (Execute): Perform arithmetic or logic operation

MEM (Memory Access): Accesses memory if needed (for load or store instructions).(*No memory access needed, proceed to WB.)*

WB (Write Back): The result of the instruction is written back to the register file.





Types of Pipeline

Pipelining is mainly classified into two types based on the nature of operations being performed. Both types aim to improve performance through parallel execution.

- **Arithmetic Pipeline**
- **Instruction Pipeline**

Arithmetic Pipeline

An arithmetic pipeline divides an arithmetic problem into various sub problems for execution in various pipeline segments. It is used for floating point operations, multiplication and various other computations.

Floating point addition using arithmetic pipeline

The following sub operations are performed in this case

- Exponent comparison
- Mantissa alignment(Mantissa:significant digit of floating point numbers),
- Addition/Subtraction
- Normalization

example:ADD 2 number X and Y (X=0.3214*10^3 and Y=0.4500*10^2)

First of all the two exponents are compared and the larger of two exponents is chosen as the result exponent. The difference in the exponents then decides how many times we must shift the smaller exponent to the right. Then after shifting of exponent, both the mantissas get aligned. Finally the addition of both numbers take place followed by normalisation of the result in the last segment

Instruction Pipeline

Instruction Pipeline

Instruction pipelining overlaps different phases of instruction execution such as fetch, decode, and execute. This ensures continuous utilization of CPU components.

Overlaps **IF, ID, EX stages**

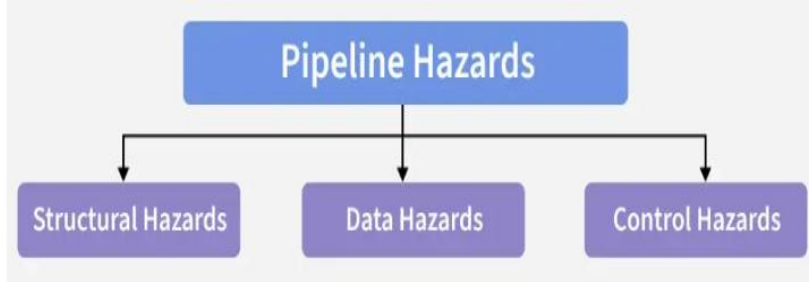
Executes multiple instructions **simultaneously**

Increases **CPU efficiency**

Common in **general-purpose processors**

Pipeline Hazards

Pipeline hazards are conditions that cause the pipeline to stall or produce incorrect results. These hazards prevent smooth execution of instructions.



- Occur due to **dependency or conflicts**
- Cause **pipeline stalls**
- Reduce performance if frequent
- Must be handled carefully

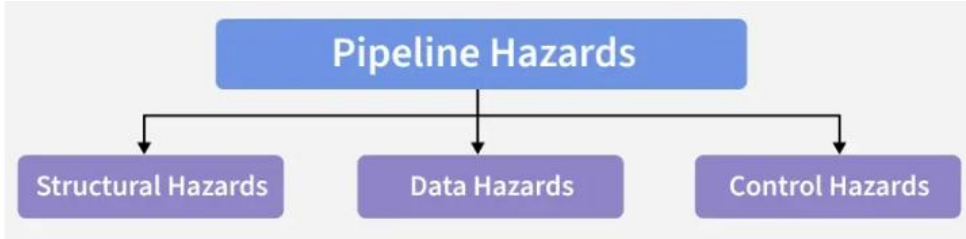
Types of Pipeline Hazards

There are different types of pipeline hazards depending on the source of the problem.

Data Hazard: A data hazard occurs when instructions exhibit data dependencies such that one instruction depends on the result of a previous instruction that has not yet completed in the pipeline

Pipeline Hazards

Control Hazard: A control hazard (also known as a branch hazard) arises when the flow of instruction execution is altered by branch or jump instructions. Since fetching the next instruction depends on the outcome of a branch, the pipeline may fetch the wrong instructions before knowing the actual branch direction



Structural Hazard: A structural hazard, also called a resource conflict, occurs when two or more instructions require access to the same hardware resource simultaneously, and the hardware cannot support the required parallel access

Advantages and Disadvantages of Pipelining

Advantages of Pipelining

Pipelining significantly improves system performance by increasing instruction throughput and better utilization of hardware resources.

- Higher instruction throughput
- Reduced processor cycle time
- Efficient use of CPU components
- Supports parallel processing

Disadvantages of Pipelining:pipelining also has some limitations related to complexity and control.

- Complex processor design
- Increased instruction latency
- Performance affected by branching
- Pipeline stalls reduce efficiency

Specialized Microprocessors

- **Embedded Microprocessor**
- **bit-slice microprocessor**
- **Arithmetic Microprocessor**

Embedded Microprocessor

Definition:

A microprocessor designed to perform specific control tasks within embedded systems.

Characteristics:

- Low power consumption
- Real-time operation
- Small size and cost-efficient
- Specialized instruction sets

Examples:

Intel 8051 — widely used microcontroller in embedded devices

ARM Cortex-M series — popular in modern IoT and mobile devices

Applications:

Home appliances (microwaves, washing machines)

Automotive control systems

Industrial automation

bit-slice microprocessor

A **bit-slice microprocessor** is a CPU design method that builds a processor by combining smaller "slice" modules—typically 4-bit Arithmetic Logic Units (ALUs)—to form a CPU of desired word length (like 8-bit, 16-bit, 32-bit, or more).

How It Works:

The processor is constructed from multiple identical bit-slice modules connected in parallel.

Each module processes a portion (slice) of the full data word (e.g., 4 bits).

Control logic manages the coordination between slices to execute operations across the full word size

Advantages:

Flexible CPU word size

Enables custom control logic

Good for specialized or high-performance CPUs

Examples:

AMD 2901 — 4-bit slice ALU used to build 16- or 32-bit processors

Arithmetic Microprocessor

What is an Arithmetic Microprocessor?

Specialized microprocessor or unit focused on arithmetic operations

Core components:

- **Arithmetic Logic Unit (ALU):** Performs addition, subtraction, and logic operations
- **Floating Point Unit (FPU):** Handles floating-point calculations (decimals)

Handles complex math operations like multiplication, division, square root

Essential for scientific, multimedia, and engineering applications

Importance:

- Speeds up mathematical computations compared to software-only methods
- Critical in digital signal processing, graphics, scientific simulations, cryptography

Examples:

- **Intel 8087:** Early math coprocessor for floating-point operations